

Generative Pre-Trained music generator

Miguel García Bohigues

Enero 2023

1 Introducción

La industria de la música ha ido evolucionando a lo largo del tiempo. Y con ella, los géneros que fueses donde fueses podías escuchar. Desde el rock & roll de los años 50, pasando por el Jazz de los 60, el heavy-metal de los 70 80, el rap de los 90, el pop de los 2000, hasta llegar al reggaeton actual.

Durante todas estas décadas, la industria se ha ido adaptando a las exigencias de la audiencia, copando prácticamente todos los medios de comunicación de canciones y artistas de los géneros dominantes, dejando una pequeña ventana para algún artista en despegue que rompe con la tónica.

A pesar de las nuevas tecnologías de música a la carta como Spotify o Deezer, entre otras, la realidad sigue siendo la misma que a principios de los 2000. Si quieres escuchar algo diferente, fuera de lo que suena en las principales cadenas, has de ir a para a artistas concretos buscando sus álbumes o su nuevo contenido.

Este problema se acrecienta cuando el género que te gusta hace tiempo que se cayó de la rueda del mainstream, como es el caso del heavy metal. Gran parte de las bandas se han retirado, o ya no sacan nuevo contenido, sino que se dedican a hacer giras con cuentagotas.

En este trabajo tenemos el objetivo ambicioso de cambiar esta realidad. De poder escuchar contenido nuevo de artistas que revolucionaron su género pero que ya no se encuentran con capacidades para hacerlo.

Siendo conscientes de la gran dimensionalidad del proyecto, y acotándolo a un alcance razonable dado el tiempo con el que contamos, nos conformaremos con poder generar de manera automática, una secuencia de notas que se asemejen a una canción del género escogido.

Para llevarlo a cabo, crearemos un corpus de canciones, lo modificaremos para que cumpla nuestras necesidades y crearemos y entrenaremos un modelo de aprendizaje automático capaz de generar esta secuencia de notas que buscamos.

2 Corpus

El primer paso para poder conseguir nuestro objetivo es obtener un corpus que contenga la información necesaria para poder trabajar. Una búsqueda rápida en internet nos llevó a the million song dataset. Un conjunto de metadatos de un millón de canciones, como su nombre indica. No obstante, aunque a priori podría valernos, el corpus pesa 280GB y su versión reducida de 1GB presenta demasiada heterogeneidad para poder ser utilizado.

En lugar de utilizar un corpus ya existente, decidimos crear el nuestro propio. Para ello, empezamos a investigar la mejor manera de representar las notas. Como primera aproximación intentamos hacerlo con *mp3* o *wav*, pero el tamaño de los ficheros era muy grande. Mirando posibles alternativas, encontramos la codificación *MIDI*.

Musical Instrument Digital Interface o por sus siglas MIDI, es un estándar que permite que varios instrumentos y dispositivos se relacionen y comuniquen entre sí. Se estandarizó en los 80 y se empezó a utilizar en la mayoría de estudios

de grabación. En esencia, MIDI codifica el tono, duración y velocidad de una nota. Además de todo esto, MIDI es un formato realmente ligero, siendo que una canción de 7 minutos sólo ocupa 70kB.

Con el formato decidido, sólo falta conseguir las canciones. Para ello se ha escogido uno de los grupos más influyentes del Heavy Metal internacional, Metallica. Para descargar las canciones en dicho formato se ha realizado un crawler que ha recorrido la página free midi, descargando todas las canciones del artista.

De esta manera hemos generado un repositorio con las 110 canciones que el sitio tiene disponible para descargar. O lo que es lo mismo, un total de casi 2 millones de notas y 75 instrumentos diferentes.

3 Entorno y herramientas

Para el desarrollo del modelo, se ha utilizado python junto con las librerías de aprendizaje automático dadas en clase: keras y tensorflow.

Para el tratamiento en python de los ficheros midi, se ha utilizado una librería llamada pretty midi la cual se ha extendido para poder ajustarse a nuestras necesidades.

Pretty midi presenta una clase Note, que contiene la información básica de una nota (tono, velocidad, inicio...) pero carece de información sobre el instrumento que la toca. Es por esto que hemos extendido la clase Note a una clase INote, donde también aparece la información de este último.

Remarcar que, esta información no nos sería necesaria si no concebiésemos la posibilidad de más de un instrumento en la secuencia que generemos.

Todo el desarrollo de los modelos se ha realizado sobre la plataforma de google colab debido a que el hardware del que disponemos hubiese limitado mucho la solución a planear.

4 Primera aproximación: LSTM

Com primera aproximación a la solución, pensamos en la utilización de unidades LSTM (long-short term memory), debido a su idoneidad para el tratamiento de secuencias.

4.1 Representación

Las notas, en una canción, son equivalentes a las palabras dentro de una frase, son secuencias sonoras que por separado carecen de sentido, pero que cuando las juntas ofrecen una locofonía comprensible.

Por ello, nuestra primera aproximación se corresponde con la generación de cadenas (listas) de notas que componen las canciones. Para poder trabajar mejor con los datos, creamos una tabla con:

- Instrumento

- Velocidad
- Tono
- Momento de inicio
- Momento final
- Duración

Para reducir la dimensionalidad del problema, sólo hemos considerado los valores de tono (pitch) que oscilan entre 1 y 128, puesto que son los soportados por la mayoría de instrumentos. Además, de los 75 instrumentos originales, hemos eliminado aquellos cuyo número de notas no superaba las 1000, quedándonos con 25 y reduciendo así el número de clases a 1/3 del original.

4.2 Arquitectura de la solución

Como arquitectura del modelo, instanciamos una capa de entrada bidimensional de $seq_len * num_feat$. Es decir, el número de caracteres que al mismo tiempo van a entrar a la LSTM y el número de características a tener en cuenta.

```

1 input_shape = (SEQ_LENGTH, 3)
2
3 inputs = tf.keras.Input(input_shape)
4 x = tf.keras.layers.LSTM(256)(inputs)
5
6 outputs = {
7     'pitch': tf.keras.layers.Dense(128, name='pitch')(x),
8     'duration': tf.keras.layers.Dense(1, name='duration')(x),
9     'instrument': tf.keras.layers.Dense(25, name='instrument')(x)
10 }
11
12 model = tf.keras.Model(inputs, outputs)
13
14 loss = {
15     'pitch': tf.keras.losses.SparseCategoricalCrossentropy(
16         from_logits=True),
17     'duration': tf.keras.losses.SparseCategoricalCrossentropy(
18         from_logits=True)
19 }
20
21 optimizer = tf.keras.optimizers.Adam(learning_rate=LEARNING_RATE)
22
23 model.compile(
24     loss=loss,
25     loss_weights={
26         'pitch': 0.05,
27         'duration': 1.0,
28         'instrument': 0.1
29     },
30     optimizer=optimizer,

```

Además, la LSTM cuenta con 3 capas densas, una por cada salida. Cada una de ellas cuenta con una neurona por cada posible clase (valor) que puede tomar como salida: 128 para el pitch, 1 para la duración y 25 para los instrumentos.

Como funciones de pérdida se ha utilizado *SparseCategoricalCrossEntropy* y una variante del error cuadrático con presión positiva para evitar que la LSTM genere valores negativos. Además, vemos como las funciones de pérdida estaban ponderadas para igualar las magnitudes.

4.3 Resultados

Los resultados del modelo no han sido otra cosa que decepcionantes. Cómo vemos en la figura 1, a priori el modelo ha conseguido aprender el conjunto de entrenamiento,

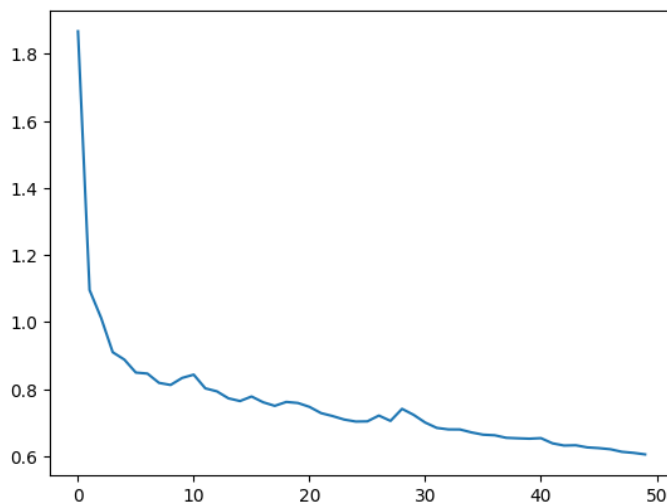


Figure 1: Evolución de la pérdida

Sin embargo, a la hora de generar audio, lo que mayoritariamente se escucha es silencio. Echando un vistazo a los datos generados (tabla 1) vemos como la mayoría de ellos tienen duración 0 o realmente baja. Además, vemos cómo las notas están muy juntas entre sí, no dejando espacio para la comprensión y resultando puramente en ruido.

A pesar de los malos resultados, sacamos conclusiones positivas de esta aproximación, por ello la hemos reflejado en la memoria. Más allá del conocimiento adquirido al haber trabajado con LSTM y haber modificado nuestra propia función de pérdida, hemos recaído en un detalle que no habíamos tenido en cuenta.

Nuestra representación inicial no era correcta. Estábamos asumiendo una secuencialidad en las notas, por lo que si en un mismo momento se da una nota de bajo, una de batería y una de guitarra, las estábamos computando como tres

note Id	pitch	duration	instrument	start	end
458	40	0.000000	0	2.599751	2.599751
459	66	0.000000	0	2.597698	2.597698
460	28	0.000000	6	2.599632	2.599632
461	55	0.000000	0	2.608192	2.608192
462	76	0.001359	6	2.608625	2.609984
463	39	0.000000	0	2.602117	2.602117
464	45	0.000399	0	2.606268	2.606667
465	117	0.000000	0	2.605557	2.605557
466	79	0.087687	3	2.609834	2.697521
467	98	0.071042	0	2.703197	2.774239
468	34	0.159814	6	2.782897	2.942711
469	36	0.000000	0	2.933622	2.933622
470	74	0.015653	0	2.940813	2.956466

Table 1: Extracto de notas de la mitad de la secuencia generada

notas en tres momentos diferentes, no 3 notas en el mismo momento. Fruto de esto se da el caos de secuencialidad en las notas y la poca duración de estas.

5 GPT

En esta sección vamos a comentar las modificaciones y desarrollos realizados para la generación de música con modelos GPT.

5.1 Representación

La motivación de utilizar modelos GPT, nace de la idea de generación de texto. En secciones anteriores hemos mencionado que las notas y las palabras no son tan diferentes. Entonces, ¿Porqué no representarlas de igual manera?

Siguiendo esta idea, hemos transcrito las canciones. Para ello hemos definido una ventana de tiempo en el que puede sonar una nota, y hemos corrido la ventana a lo largo de toda la canción, codificando las notas que suenan en paralelo.

De esta manera, hemos codificado las notas como: “tono.instrumento” seguido de un espacio. La última nota que suene en la ventana, tendrá el carácter especial ‘\$’. de esta manera podemos representar la concurrencia de las notas y aprovechar la potencia de los modelos GPT para texto.

Un ejemplo de un verso sería:

```
1 47.0_1.0 47.0_1.0$ 47.0_1.0$ 47.0_1.0$ 47.0_1.0$ 47.0_1.0$
```

Partiendo de esta representación, para realizar las pruebas hemos partido de un corpus reducido, en un principio con sólo dos álbumes de la banda. Esto hace un total de 18 canciones.

5.2 Arquitectura

Tal y como se lleva comentando en toda esta sección de la memoria, la arquitectura utilizada se corresponde con un modelo GPT. Es decir, el decoder de un transformer pero sin el componente de auto-atención que recibe la entrada del encoder. En la figura 2 se puede ver una representación extraída de las diapositivas de clase.

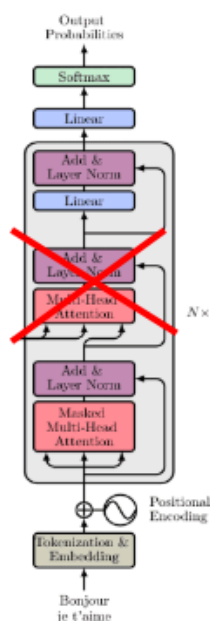


Figure 2: Arquitectura del modelo GPT

La implementación la hemos hecho completamente a mano, A través de las capas de abstracción que nos proporcionan tanto *keras* como *tensorflow*. De esta manera hemos definido la capa de atención enmascarada, la capa FeedForward, la capa de codificación y del embedding.

Tras la implementación, ha habido dos factores que han generado muchos problemas, fruto seguramente de la poca experiencia con los toolkits. El primero ha sido la carga del dataset en el entorno. Para ello, tras mucho investigar, se ha utilizado una utilidad de *tensorflow*: *TextLineDataset*, que nos permite cargar directamente en dicho formato las líneas de un texto. Tras hacer esto sólo quedaba separar el conjunto en batches y aplicar las etiquetas. Para nuestro caso, la etiqueta de una palabra no es más que el código dentro del vocabulario de la palabra que va inmediatamente después. De esta manera hemos construido un dataset de tuplas (valor, etiqueta) para poderlo pasar al modelo.

Otro de los aspectos con los que hemos tenido que pelear han sido los hiperparámetros que regulan la dimensión del modelo GPT. Una pequeña variación

en el tamaño del batch o de las dimensiones de la capa feed-forward, podía hacer que el modelo arrojase un error de “Out of memory” o que tuviesemos un problema de desvanecimiento del gradiente y nuestra pérdida fuese 0.

Tras mucha experimentación, los parámetros con los que nos hemos quedado han sido:

- vocabulario de 600 palabras máximo
- tamaño máximo de la secuencia de entrada de 64
- dimensiones del embedding y de la feed-forward de 128
- 2 cabezas
- Y un factor de dropout del 30%

Con ello hemos entrenado nuestro modelo durante 50 épocas. obteniendo una convergencia en la pérdida.

Aunque en este punto hay que tener cuidado, puesto que un sobre-entrenamiento hace que el modelo genere siempre una secuencia de palabras que se corresponde con las que mayor frecuencia tienen en el dataset.

5.3 Resultados

En este caso los resultados sí han sido muy satisfactorios. Tras entrenar el sistema, se ha escogido una secuencia pseudo-arbitraria y se la ha pasado como entrada al modelo para que genere más a partir de ella. En el siguiente fragmento de código se muestra una parte de toda la secuencia generada:

```
1 '72.0_11.0', '72.0_11.0', '72.0_11.0', '72.0_11.0',  
2 '72.0_11.0', '28.0_7.0$', '54.0_6.0', '69.0_8.0',  
3 '70.0_5.0$', '34.0_7.0', '50.0_5.0', '38.0_0.0$',  
4 '36.0_0.0$', '42.0_0.0', '52.0_6.0', '28.0_7.0',  
5 '42.0_0.0', '52.0_6.0', '28.0_7.0', '42.0_0.0',  
6 '52.0_6.0', '28.0_7.0', '42.0_0.0', '52.0_6.0',  
7 '28.0_7.0', '42.0_0.0', '52.0_6.0', '28.0_7.0', '42.0_0.0',  
8 '52.0_6.0', '28.0_7.0', '42.0_0.0', '52.0_6.0',  
9 '28.0_7.0', '42.0_0.0', '52.0_6.0', '28.0_7.0',  
10 '42.0_0.0', '52.0_6.0', '28.0_7.0', ...
```

Como vemos, hay bastante variedad en las notas, a pesar de que la cadena de entrada era una secuencia que empezaba por 5 notas iguales y 3 diferentes, el sistema ha sabido seguir con el ritmo generando una secuencia muy similar al heavy metal.

Durante la evaluación de las secuencias, se buscaba un símil bueno con el que poder equiparar el sonido del MIDI, puesto que acostumbrados a escuchar la señal decodificada, escucharla en esta codificación puede resultar en una impresión peor de la que realmente es. Por ello se ha buscado un equivalente. Este es At Doom's gates el tema principal de una de las mejores bandas sonoras que se han creado para un videojuego y que bebe directamente de este género.

En el siguiente enlace se puede descargar una canción generada por nuestro modelo para poderla comparar con el video anterior.

6 Conclusiones

La generación de música es una tarea muy compleja que requiere de muchos aspectos a tener en cuenta: el tempo, qué instrumentos van a sonar, qué notas van a tocar, la letra de la canción... No obstante, en el alcance que habíamos dado al trabajo consideramos que hemos conseguido el objetivo.

Además de haber aprendido mucho sobre las herramientas actuales de desarrollo de modelos de aprendizaje automático, hemos reforzado conceptos teóricos al realizar las implementaciones manualmente y no utilizar capas de abstracción muy elevadas.

No obstante, somos conocedores de las limitaciones del modelo y de sus posibles mejoras: como el rápido sobre-ajuste.